School of
Computing and
Information Systems

SMU
SINGAPORE MANAGEMENT UNIVERSITY

Collaborative Software Development

Group Project

None Of Us Is As Smart As All Of Us

Ken Blanchard

Project Teams

- Self-formation of 5-6 individuals into a team
  - Instructors to place students without a team into a random team.
  - Finalize team formation before the end of 1st week.
- Enroll to groups on eLearn.
- No switching of teams allowed after the 1st week.

Theme: Curated Self-Learning for Gen-Alpha Culture

Build a self-learning platform that helps users understand and engage with Gen-Alpha culture (e.g., language, online practices, social references) through curated, high-quality learning content. The platform should emphasize intentional learning and reflection, rather than low-effort or

2025 Word of the Year: Slop

Plus 'gerrymander', 'touch grass', 'performative', and other words that defined the year

14 Dec 2025

Slop

Merriam-Webster's human editors have chosen *slop* as the 2025 Word of the Year. We define *slop* as "digital content of low quality that is produced usually in quantity by means of artificial intelligence." All that stuff dumped on our screens, captured in just four letters: the English language came through again.

2025 Word of the Year Is "Slop"

January 9, 2026

New Orleans Marriott–Jan. 9—The American Dialect Society, in its 36th annual words-of-the-year vote, selected *slop* as the Word of the Year for 2025. More than three hundred attendees took part in the deliberations and voting, in an event hosted in conjunction with the Linguistic Society of America's annual meeting.

The word *slop* was recognized for its widespread use for low-quality, high-quantity content, most typically produced by generative AI. While *AI slop* was a nominee in the American Dialect Society's 2024 Word of the Year vote, in 2025 *slop* could stand on its own, with the AI context often implicitly understood. *Slop* was also recognized as a productive combining form to describe anything of little value generated in mass quantities.

Theme: Curated Self-Learning for Gen-Alpha Culture (core)

- User accounts and profiles
  - Registration, authentication, and basic profile management
  - Role differentiation (e.g., learner, contributor, moderator/admin)
- Learning content management
  - CRUD functionality for cultural concepts / explainers
  - CRUD functionality for lessons composed of multiple concepts
  - Lessons should define clear learning objectives and include quizzes or challenges
- Learning progress tracking
  - Lesson enrollment and completion tracking
  - Quiz results and basic learning feedback
  - Simple indicators such as completed lessons or streaks
- Content quality and moderation
  - Workflow for submitting, reviewing, approving, or rejecting content
  - Ability for users to flag low-quality or misleading content
  - Moderation features to explicitly counter “AI slop”
- Backend system
  - RESTful API with documented endpoints (e.g., Swagger / OpenAPI)
  - Persistent storage for users, content, progress, and moderation data
  - Public deployment of the backend on a cloud platform

Theme: Curated Self-Learning for Gen-Alpha Culture (extend)

Extensions (choose at least one):

- Online, AI-positive self-learning features
  - Guardrailed AI assistance for drafting or explaining concepts
  - Personalized learning paths or recommendations
  - Multimedia-heavy learning objectives (e.g., video-linked lessons)

- Offline, action-based self-learning features
  - Offline learning activities or “quests” that encourage human interaction
  - Buddy or group-based challenges
  - Reflection or check-in mechanisms to record offline learning outcomes

Project Grading

- Agile process
  - Team to decide on stories, sprints, backlog, tracking, etc.
  - Team to assign own roles as project owner, Scrum master, etc.

- Tools
  - Java, Spring Boot (mandatory), and other languages/frameworks as needed
  - Git (mandatory): Bitbucket/GitHub
  - Project management and tracking: Jira (recommended)

- Public deployment of services
  - Team to choose deployment platforms (AWS, Azure, etc.)

Project Grading

- Baseline performance:
  - Working prototype with must-have features stated in the core slide + one extension.
  - Evidence of software process and development practices as covered in the course.

- Beyond baseline:
  - Must justify how the additional feature is useful in the report/presentation.
  - +0.5 grade from baseline for fully functional additional features implemented. For example:
    - Learning analytics
    - Social learning features
    - Trust and credibility signals
    - Web or mobile user interface

- Note that extra technical features are not counted for grading:
  - You can implement them if you need, e.g. load balancing, microservices, full CD pipeline, enhanced security, etc.

SMU SINGAPORE MANAGEMENT UNIVERSITY

Lesson Plan

Week	Topic	Assessment
1	Software processes	
2	User stories, estimations and planning	
3	Collaboration methods	Project vision (not graded)
4	Software design	
5	Persistence	
6	Sprint execution	
7	Sprint review and retrospective (midterm)	Demo and document
9	Security	
10	Software testing	
11	Code review, refactoring, pair programming	
12	Automation and deployment	
13	Sprint review and retrospective (final)	Demo and document

Project Grading (35%)

- Midterm demo and document (week 7, 10%)
  - Expected: at least 1 sprint completed, working demo
    - For the demo, the following should be implemented and demonstrated:
      - API routes with Swagger UI, business logic, and persistence.
    - Follow sound, relevant software process, design and implementation principles (at least what are covered in the class up to this point).
    - Document: software process and technical details up to this point (strict limit of 4 pages).
      - See the last 2 slides of this deck for what kind of content should be included.
      - Submission deadline: one week after the in-class demo.
    - Source code shared via git repository (provide a link in the submission doc).
    - 10-min software demo per team during class time.
  - Extras: any fully functional feature beyond the expected.

Project Grading (35%)

- Final demo (25%)
  - Demo (20 mins): week 13, in class
    - Functional app
  - Submission: after the demo
    - 1 document (limit of 8 pages: software process and technical details)
    - Source code (link to git repository in the doc)

- Peer-evaluation
  - Team member to rate the contribution of others
    - Proper justification needed for each rating
    - Individual project grades will be heavily moderated based on the peer-eval results.

Note that the page limit for all submissions does not apply to the table of content and cover page for the documents. No appendix allowed. All documents to be submitted on eLearn in PDF format only.

Document Submission (1): Software Process

- Capture the agile process implemented
  - Representative screenshots needed, with important items highlighted and explained
- Capture key details on the project backlog
  - User story organization
  - How the backlog evolved over the course of the project
- Capture key details for selected sprints
  - Sprint backlog
  - User story details and estimation
  - Sprint meeting/code review
  - Planning and progress tracking
  - Sprint retrospective, lesson learnt, etc.
- Evidence of collaboration workflows using Git
- Statistics obtained from Git repository for each team member: e.g., percentage of commits for everyone, etc. It's up to the team to include other measures of individual contribution.
- Details on any other things not listed here

SMU SINGAPORE MANAGEMENT UNIVERSITY

Document Submission (2): Technical Details

- Software design: component based architecture, services, etc.
- API route design and implementation

- User interface design if any
- Database design

- Security design
- Details on testing

- Details on continuous integration, deployment strategies, etc.
- Any other things beyond what listed here